COMPUTER ENGINEERING
QUEZON CITY

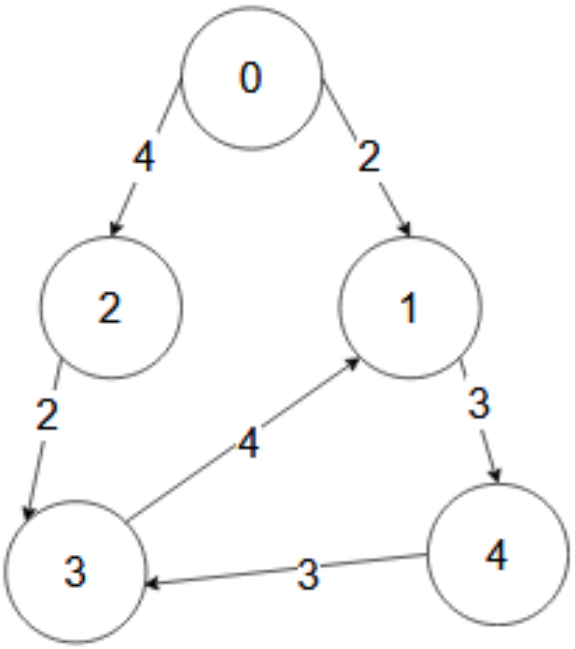
COURSE CODE/TITLE:	CPE 010 - Data Structure Algorithms	**SECTION:**	CpE21S2
DATE PERFORMED:	10/02/2026	**DATE SUBMITTED:**	10/02/2026
NAME:	Esguerra, Paula	**INSTRUCTOR:**	Engr. Jimlord Quejado

ASSIGNMENT 10 Graphs

1. PROCEDURE OUTPUT

ILO A: Create C++ code for graph implementation utilizing adjacency matrix and adjacency list

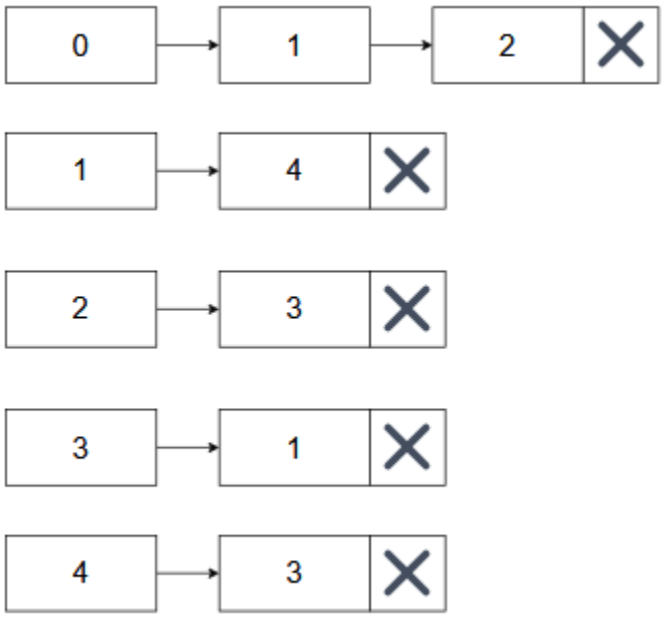
A.1. Create a Graph



A.2. Adjacency Matrix

	0	1	2	3	4
0	0	2	4	0	0
1	0	0	0	0	3
2	0	0	0	2	0
3	0	4	0	0	0
4	0	0	0	3	0

A.3. Adjacency List



Section 6: (code)

Code Screenshot

```
#include <iostream>

// stores adjacency list items
struct adjNode {
    int val, cost;
    adjNode* next;
};

// structure to store edges
struct graphEdge {
    int start_ver, end_ver, weight;
};

class DiaGraph{
    // insert new nodes into adjacency list from given graph
    adjNode* getAdjListNode(int value, int weight, adjNode* head) {
        adjNode* newNode = new adjNode;
        newNode->val = value;
        newNode->cost = weight;
        newNode->next = head; // point new node to current head
        return newNode;
    }
```

```

    int N; // number of nodes in the graph
    public:
    adjNode **head; //adjacency list as array of pointers

    // Constructor
    DiaGraph(graphEdge edges[], int n, int N) {
        // allocate new node
        head = new adjNode*[N]();
        this->N = N;
        // initialize head pointer for all vertices
        for (int i = 0; i < N; ++i)
        head[i] = nullptr;
        // construct directed graph by adding edges to it
        for (unsigned i = 0; i < n; i++) {
            int start_ver = edges[i].start_ver;
            int end_ver = edges[i].end_ver;
            int weight = edges[i].weight;
            // insert in the beginning
            adjNode* newNode = getAdjListNode(end_ver, weight, head[start_ver]);
            // point head pointer to new node
            head[start_ver] = newNode;
        }
    }

    // Destructor
```

```
    ~DiaGraph() {
        for (int i = 0; i < N; i++)
        delete[] head[i];
        delete[] head;
        }
    };

// print all adjacent vertices of given vertex
void display_AdjList(adjNode* ptr, int i){
    while (ptr != nullptr) {
        std::cout << "(" << i << ", " << ptr->val
        << ", " << ptr->cost << ") ";
        ptr = ptr->next;
        }
    std::cout << std::endl;
    }

// graph implementation
int main(){
    // graph edges array.
    graphEdge edges[] = {
        // (x, y, w) -> edge from x to y with weight w
        {0,1,2},{0,2,4},{1,4,3},{2,3,2},{3,1,4},{4,3,3}
        };
    int N = 6; // Number of vertices in the graph

    // calculate number of (unsigned long long)12Ui64
    int n = sizeof(edges)/sizeof(edges[0]);

    // construct graph
    DiaGraph diagraph(edges, n, N);

    // print adjacency list representation of graph
      std::cout<<"Graph adjacency list "<<
      std::endl<<"(start_vertex, end_vertex,weight):"<<std::endl;

      for (int i = 0; i < N; i++){
          // display adjacent vertices of vertex i
          display_AdjList(diagraph.head[i], i);
          }
      return 0;
      }

```

Output

```
Graph adjacency list
(start_vertex, end_vertex,weight):
(0, 2, 4) (0, 1, 2)
(1, 4, 3)
(2, 3, 2)
(3, 1, 4)
(4, 3, 3)
```

B.1. Depth-First Search

Code Screenshot

```cpp
#include <string>
#include <vector>
#include <iostream>
#include <set>
#include <map>
#include <stack>

template <typename T>
class Graph;

template <typename T>
struct Edge{
    size_t src;
    size_t dest;
    T weight;
    // To compare edges, only compare their weights,
    // and not the source/destination vertices
    inline bool operator<(const Edge<T> &e) const
    {
        return this->weight < e.weight;
    }
    inline bool operator>(const Edge<T> &e) const
    {
        return this->weight > e.weight;
    }
};

template <typename T>
std::ostream &operator<<(std::ostream &os, const Graph<T> &G){
    // Changed to '<=' because your vertices use 1-based indexing up to V
    for (auto i = 1; i <= G.vertices(); i++){
        os << i << ":\t";
        auto edges = G.outgoing_edges(i);
        for (auto &e : edges)
            os << "{" << e.dest << ": " << e.weight << "}, ";
        os << std::endl;
    }
    return os;
}

template <typename T>
class Graph{
    public:
    // Initialize the graph with N vertices
    Graph(size_t N) : V(N){
    }
    // Return number of vertices in the graph
    size_t vertices() const{
```

```
49          return V;
50      }
51
52      // FIX: Changed return type from 'size_t&' to 'const std::vector<Edge<T>>&'
53      const std::vector<Edge<T>>& edges() const{
54          return edge_list;
55      }
56
57      void add_edge(Edge<T> &&e){
58          // Check if the source and destination vertices are within range
59          if (e.src >= 1 && e.src <= V &&
60              e.dest >= 1 && e.dest <= V)
61              edge_list.emplace_back(e);
62          else
63              std::cerr << "Vertex out of bounds" << std::endl;
64      }
65
66      // FIX: Changed return type from 'size_t' to 'std::vector<Edge<T>>'
67      std::vector<Edge<T>> outgoing_edges(size_t v) const{
68          std::vector<Edge<T>> edges_from_v;
69          for (auto &e : edge_list){
70              if (e.src == v)
71                  edges_from_v.emplace_back(e);
72          }
73          return edges_from_v;
74      }
```

```
75
76      // Overloads the << operator so a graph be written directly to a stream
77      template <typename U>
78      friend std::ostream &operator<<(std::ostream &os, const Graph<U> &G);
79
80      private:
81          size_t V; // Stores number of vertices in graph
82          std::vector<Edge<T>> edge_list;
83  };
84
85  // FIX: Changed return type from 'size_t' to 'std::vector<size_t>'
86  template <typename T>
87  std::vector<size_t> depth_first_search(const Graph<T> &G, size_t dest){
88      std::stack<size_t> stack;
89      std::vector<size_t> visit_order;
90      std::set<size_t> visited;
91      stack.push(1); // Assume that DFS always starts from vertex ID 1
92      while (!stack.empty())
93      {
94          auto current_vertex = stack.top();
95          stack.pop();
96          // If the current vertex hasn't been visited in the past
97          if (visited.find(current_vertex) == visited.end())
98          {
99              visited.insert(current_vertex);
100             visit_order.push_back(current_vertex);
```

```
                for (auto e : G.outgoing_edges(current_vertex))
                {
                    // If the vertex hasn't been visited, insert it in the stack.
                    if(visited.find(e.dest) == visited.end())
                    {
                        stack.push(e.dest);
                    }
                }
            }
        }
        return visit_order;
}

// FIX: Changed return type from 'size_t' to 'Graph<T>'
template <typename T>
Graph<T> create_reference_graph()
{
    Graph<T> G(9);
    std::map<unsigned, std::vector<std::pair<size_t, T>>> edges;

    // Explicitly target each map key so they don't overwrite each other
    edges[1] = {{2, 0}, {5, 0}};
    edges[2] = {{1, 0}, {5, 0}, {4, 0}};
    edges[3] = {{4, 0}, {7, 0}};
    edges[4] = {{2, 0}, {3, 0}, {5, 0}, {6, 0}, {8, 0}};
    edges[5] = {{1, 0}, {2, 0}, {4, 0}, {8, 0}};
    edges[6] = {{4, 0}, {7, 0}, {8, 0}};
    edges[7] = {{3, 0}, {6, 0}};
    edges[8] = {{4, 0}, {5, 0}, {6, 0}};

    for (auto &i : edges)
        for (auto &j : i.second)
            G.add_edge(Edge<T>{i.first, j.first, j.second});
    return G;
}

template <typename T>
void test_DFS(){
    // Create an instance of and print the graph
    auto G = create_reference_graph<T>(); // Changed explicitly to <T> for consistency
    std::cout << G << std::endl;
    // Run DFS starting from vertex ID 1 and print the order
    // in which vertices are visited.
    std::cout << "DFS Order of vertices: " << std::endl;
    auto dfs_visit_order = depth_first_search(G, 1);
    for (auto v : dfs_visit_order)
        std::cout << v << std::endl;
}

int main(){
    using T = unsigned;
    test_DFS<T>();
    return 0;
}
```

Output

```
D:\2nd yr\CPE 010_C++\Graph
1:      {2: 0}, {5: 0},
2:      {1: 0}, {5: 0}, {4: 0},
3:      {4: 0}, {7: 0},
4:      {2: 0}, {3: 0}, {5: 0}, {6: 0}, {8: 0},
5:      {1: 0}, {2: 0}, {4: 0}, {8: 0},
6:      {4: 0}, {7: 0}, {8: 0},
7:      {3: 0}, {6: 0},
8:      {4: 0}, {5: 0}, {6: 0},
9:

DFS Order of vertices:
1
5
8
6
7
3
4
2


--------------------------------
Process exited after 0.264 seconds with return value 0
Press any key to continue . . .
```

B.2. Breadth-First Search

Code ScreenShot

```cpp
#include <string>
#include <vector>
#include <iostream>
#include <set>
#include <map>
#include <queue>

template <typename T>
class Graph;

template <typename T>
struct Edge{
    size_t src;
    size_t dest;
    T weight;

    inline bool operator<(const Edge<T> &e) const{
        return this->weight < e.weight;
    }
    inline bool operator>(const Edge<T> &e) const{
        return this->weight > e.weight;
    }
};

// --- ADDED: The missing Graph class declaration so the code compiles ---
```

```cpp
template <typename T>
class Graph {
public:
    Graph(size_t N) : V(N) {}
    size_t vertices() const { return V; }

    void add_edge(Edge<T> &&e) {
        if (e.src >= 1 && e.src <= V && e.dest >= 1 && e.dest <= V)
            edge_list.emplace_back(e);
    }

    std::vector<Edge<T>> outgoing_edges(size_t v) const {
        std::vector<Edge<T>> edges_from_v;
        for (const auto &e : edge_list) {
            if (e.src == v) edges_from_v.emplace_back(e);
        }
        return edges_from_v;
    }
private:
    size_t V;
    std::vector<Edge<T>> edge_list;
};

template <typename T>
std::ostream &operator<<(std::ostream &os, const Graph<T> &G)
{
    // Changed to '<=' to match your 1-based index limit
    for (auto i = 1; i <= G.vertices(); i++){
        os << i << ":\t";
        auto edges = G.outgoing_edges(i);
        for (auto &e : edges)
            os << "{" << e.dest << ": " << e.weight << "}, ";
        os << std::endl;
    }
    return os; // FIX: Added missing return statement
}

// FIX: Changed 'auto' to 'Graph<T>'
template <typename T>
Graph<T> create_reference_graph()
{
    Graph<T> G(8);
    std::map<unsigned, std::vector<std::pair<size_t, T>>> edges;
    edges[1] = {{2, 2}, {5, 3}};
    edges[2] = {{1, 2}, {5, 5}, {4, 1}};
    edges[3] = {{4, 2}, {7, 3}};
    edges[4] = {{2, 1}, {3, 2}, {5, 2}, {6, 4}, {8, 5}};
    edges[5] = {{1, 3}, {2, 5}, {4, 2}, {8, 3}};
    edges[6] = {{4, 4}, {7, 4}, {8, 1}};
    edges[7] = {{3, 3}, {6, 4}};
    edges[8] = {{4, 5}, {5, 3}, {6, 1}};
    for (auto &i : edges)
        for (auto &j : i.second)
            G.add_edge(Edge<T>{i.first, j.first, j.second});
    return G;
}

// FIX: Changed 'auto' to 'std::vector<size_t>'
template <typename T>
std::vector<size_t> breadth_first_search(const Graph<T> &G, size_t dest)
{
    std::queue<size_t> queue;
    std::vector<size_t> visit_order;
    std::set<size_t> visited;
    queue.push(1); // Assume that BFS always starts from vertex ID 1
    while (!queue.empty())
    {
        auto current_vertex = queue.front();
        queue.pop();
        // If the current vertex hasn't been visited in the past
        if (visited.find(current_vertex) == visited.end())
        {
            visited.insert(current_vertex);
            visit_order.push_back(current_vertex);
            for (auto e : G.outgoing_edges(current_vertex))
                queue.push(e.dest);
        }
    }
```

```cpp
    return visit_order;
}

template <typename T>
void test_BFS()
{
    // Create an instance of and print the graph
    auto G = create_reference_graph<unsigned>();
    std::cout << G << std::endl;
    // Run BFS starting from vertex ID 1 and print the order
    // in which vertices are visited.
    std::cout << "BFS Order of vertices: " << std::endl;
    auto bfs_visit_order = breadth_first_search(G, 1);
    for (auto v : bfs_visit_order)
        std::cout << v << std::endl;
}

int main()
{
    using T = unsigned;
    test_BFS<T>();
    return 0;
}
```

Output

```
D:\2nd yr\CPE 010_C++\Graph
1:      {2: 2}, {5: 3},
2:      {1: 2}, {5: 5}, {4: 1},
3:      {4: 2}, {7: 3},
4:      {2: 1}, {3: 2}, {5: 2}, {6: 4}, {8: 5},
5:      {1: 3}, {2: 5}, {4: 2}, {8: 3},
6:      {4: 4}, {7: 4}, {8: 1},
7:      {3: 3}, {6: 4},
8:      {4: 5}, {5: 3}, {6: 1},
9:

BFS Order of vertices:
1
2
5
4
8
3
6
7


--------------------------------
Process exited after 0.3885 seconds with return value 0
Press any key to continue . . .
```

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO C: Demonstrate an understanding of graph implementation, operations and traversal methods.

1. A person wants to visit different locations indicated on a map. He starts from one location (vertex) and wants to visit every vertex until it finishes from one vertex, backtracks, and then explore other vertex from same vertex. Discuss which algorithm would be most helpful to accomplish this task.

   DFS approach is good choice, where only (in a A, B, C example) C is the only exit.

2. Identify the equivalent of DFS in traversal strategies for trees. To efficiently answer this question, provide a graphical comparison, examine pseudocode and code implementation.

   Pre-order, In-Order, and Post-order traversal, because a tree is a specilized graph without cycles, it is based on when the parent node is gone to the relative to the children.

3. In the performed code, what data structure is used to implement the Breadth First Search?

   A node is discovered and quesced exactly once because the tracking happens before pushing.

4. How many times can a node be visited in the BFS?

   When a tree node have one parent, each node will only discovered and visited only once.

Conclusion:

I have learned in this activity is the exploration of graph and tree traversal strategies, the topic concludes on Depth-First Search(DFS) and Breath First Search(BFS), using debugging and theoretical review. DFS is mostly likely be used in path-deepening and backtracking scenarios, it can be easily be compared to the orders in traversal. It also uses LIFO stack structure Meanwhile, BFS is good at discovering structural layers uniformly. It also relies on the FIFO queue, this is to manage the discovery order.

I have yet to understand the code when it errors on devc++, because I need to modify some things because it is outdated, so I had to search and debugged the errors. Other than that, if I were to learn the whole code, it is fairly easy, and debugging helps me to understand more of the code, because when I do debugging I need to search where and which part of the line is having a problem, and before I search for it, I try to think first and try some methods, this activity is very useful for me.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Idea Generation**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **I have used it for the purpose of having ideas and expanding the ideas I want to learn,**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.